

MLA-C01 Study Guide — Complete Reference

(Official Blueprint + Glossary + Service Reference)

This is an all-in-one study guide built on the official AWS exam guide structure. It has four parts:

Exam logistics

The four domains, mapped to every task statement, with teaching notes

Glossary — every ML/technical term you may meet, defined

AWS service reference — every in-scope service: what it does and when to use it

Part 1 — Exam Logistics

What it validates: building, operationalizing, deploying, and maintaining ML solutions and pipelines on AWS — an *engineering/MLOps* exam, not a data-science exam.

Target candidate: ~1 year with SageMaker + AWS for ML engineering, plus ~1 year in a related role (backend dev, DevOps, data engineer, data scientist).

Format: 65 questions — **50 scored, 15 unscored** (unscored aren't identified). 130 minutes.

Scoring: scaled 100–1,000, **passing = 720, compensatory** (no per-domain minimum; only the overall score must pass). No penalty for guessing — never leave a question blank.

Question types: multiple choice (1 of 4), multiple response (all correct of 5+), ordering (place 3–5 steps), matching (pair to 3–7 prompts), case study (one scenario, several independently-scored questions).

Domain weightings: Data Preparation **28%** · Model Development **26%** · Deployment & Orchestration **22%** · Monitoring, Maintenance & Security **24%**.

Out of scope: designing full end-to-end architectures from scratch, setting ML strategy, integrating a wide array of new tools, working deeply in two+ ML domains (NLP and CV), and model quantization / its accuracy impact.

Part 2 — Domains & Task Statements

Domain 1 — Data Preparation for ML (28%)

Task 1.1 — Ingest and store data

Data formats: CSV/JSON (row-based, readable, bulky) vs **Parquet/ORC** (columnar, cheap/fast for training) vs **Avro** (row-based, schema, good for streaming) vs **RecordIO** (packed binary for SageMaker Pipe mode). For large tabular training data, **Parquet** is the usual right answer.

Core storage/sources: S3 (default data lake), EFS (shared NFS), FSx for Lustre (high-throughput, S3-linked — fastest training loads), EBS (instance block storage, Provisioned IOPS for I/O-bound work).

Streaming: Kinesis Data Streams (real-time, custom consumers) vs Data Firehose (managed delivery to S3/Redshift/OpenSearch) vs Managed Service for Apache Flink (stream processing) vs MSK/Kafka.

Merging sources: Glue (serverless Spark ETL), Spark on EMR.

Skill focus: troubleshoot capacity/scalability; make storage decisions on cost, performance, structure.

Typical scenario: "training job is I/O-bound loading from S3" → FSx for Lustre, Pipe/FastFile mode.

Task 1.2 — Transform data and perform feature engineering

Cleaning: outlier treatment, imputation, combining, deduplication.

Feature engineering: scaling/standardization (Z-score) vs normalization (Min-Max), feature splitting, binning, log transform.

Encoding: one-hot, binary, label (implies order — careful), tokenization.

Tools: Data Wrangler (visual low-code), Glue / Glue DataBrew (no-code), Spark on EMR (heavy), Lambda/Spark (streaming transforms).

Labeling: Ground Truth (managed + active learning), Mechanical Turk (workforce).

Features: Feature Store — online (low-latency inference) vs offline (S3, training); prevents training/serving skew.

Task 1.3 — Ensure data integrity and prepare for modeling

Pre-training bias: Class Imbalance (CI), Difference in Proportions of Labels (DPL) via **Clarify**. Fix imbalance with SMOTE, resampling, class weights.

Security/compliance: encryption (KMS/TLS), classification/anonymization/masking, PII/PHI/data-residency implications; **Macie** finds PII in S3; **Lake Formation** governs the lake.

Data quality: Glue DataBrew, Glue Data Quality.

Bias in data: selection & measurement bias (Clarify); reduce prediction bias via splitting, shuffling, augmentation (never shuffle time-series across the time boundary → leakage).

Loading for training: EFS / FSx.

Domain 2 — ML Model Development (26%)

Task 2.1 — Choose a modeling approach

Algorithm fit: map problem → classification/regression/clustering/forecasting/anomaly/recommendation → built-in algorithm (XGBoost, Linear Learner, K-Means, PCA, DeepAR, Random Cut Forest, BlazingText, Object Detection, Factorization Machines, K-NN).

AI services (build vs buy): Translate, Transcribe, Rekognition, Comprehend, Textract, Polly, Lex, Kendra, Personalize, Forecast, Fraud Detector, Bedrock. Prefer these when the scenario says "least operational overhead / no ML expertise."

Interpretability: simpler models are more explainable; Clarify (SHAP) gives post-hoc explanations.

Templates/FMs: JumpStart (pretrained + templates), Bedrock (foundation models). Weigh cost too.

Task 2.2 — Train and refine models

Process: epoch, step, batch size.

Faster training: early stopping, distributed training (data- vs model-parallel), Managed Spot Training + checkpointing.

Regularization: dropout, weight decay/L2, L1 (sparse → feature selection).

Tuning: AMT — random, Bayesian, Hyperband, grid; hyperparameter effects (trees/depth, layers/learning rate).

Skills: built-in algorithms, script mode (TF/PyTorch), BYOC; fine-tune via Bedrock/JumpStart; prevent overfitting/underfitting/catastrophic forgetting; ensemble/stack/boost; reduce model size (pruning, compression); version with Model Registry.

Task 2.3 — Analyze model performance

Metrics: confusion matrix, accuracy (weak on imbalance), precision (FP cost), recall (FN cost — fraud/disease), F1 (imbalance), ROC/AUC (ranking); RMSE (regression). Precision-vs-recall & threshold tuning are prime material.

Fit: baselines; overfitting vs underfitting; convergence issues (often learning rate).

Tools: Clarify (bias + SHAP), Model Debugger (convergence/gradients), Experiments (repro runs), shadow vs production variant comparison.

Domain 3 — Deployment & Orchestration (22%)

Task 3.1 — Select deployment infrastructure

Four inference options: real-time (low latency, steady, <6 MB, <60s) · serverless (spiky, scale-to-zero, cold starts, no GPU) · asynchronous (large payloads ≤1 GB, long runtime ≤1 hr, queued) · batch transform (offline, no endpoint).

Knowledge: versioning/rollback, real-time vs batch serving, CPU vs GPU, provided vs custom containers, Neo for edge.

Skills: perf/cost/latency tradeoffs; compute choice; orchestrator (Pipelines vs Airflow/MWAA); multi-model vs multi-container; target (SageMaker endpoint, ECS, EKS, Kubernetes, Lambda).

Task 3.2 — Create and script infrastructure

Knowledge: on-demand vs provisioned, scaling-policy comparison, IaC (CloudFormation vs CDK), containerization, endpoint auto scaling (demand/scheduled).

Skills: auto scaling, Spot, Lambda-behind-endpoint; cross-stack provisioning (CFN/CDK); containers (ECR/ECS/EKS/BYOC); endpoints in a VPC; SageMaker SDK deploys; scaling metrics (model latency, CPU, InvocationsPerInstance).

Task 3.3 — CI/CD orchestration

Knowledge: CodePipeline/CodeBuild/CodeDeploy, Git basics, CI/CD in ML, blue/green vs canary vs linear, repos + pipelines.

Skills: configure/troubleshoot Code* stages; Gitflow/GitHub Flow; automate build/deploy; trigger jobs with EventBridge/Pipelines/CodePipeline; add unit/integration/e2e tests; build automated retraining loops.

Domain 4 — Monitoring, Maintenance & Security (24%)

Task 4.1 — Monitor model inference

Knowledge: data drift vs concept drift; monitor data quality & performance; ML Lens principles.

Skills: Model Monitor (data quality, model quality [needs labels], bias drift, feature-attribution drift); Clarify for distribution changes; A/B testing; workflow anomaly detection.

Task 4.2 — Monitor & optimize infrastructure and costs

Knowledge: KPIs (utilization, throughput, availability, scalability, fault tolerance); X-Ray, CloudWatch Lambda/Logs Insights; CloudTrail; instance families (memory/compute/general/inference-optimized); cost tools (Cost Explorer, Budgets, Trusted Advisor); tagging.

Skills: CloudWatch Logs/alarms; CloudTrail trails; dashboards (QuickSight/CloudWatch); EventBridge monitoring; right-sizing (Inference Recommender, Compute Optimizer); tagging strategy; capacity

(provisioned concurrency, quotas, auto scaling); budgets; purchasing (Spot, On-Demand, Reserved, Savings Plans).

Task 4.3 — Secure AWS resources

Knowledge: IAM roles/policies/groups, bucket policies, Role Manager; SageMaker security/compliance; network controls; CI/CD security.

Skills: least-privilege to ML artifacts (key lever: scope `iam:PassRole` to one execution role); IAM for users *and* apps (distinguish calling **principal** from **execution role**); monitor/audit/log (CloudTrail + CloudWatch); troubleshoot (S3-access failure = execution-role or KMS gap); VPCs/subnets/security groups, VPC mode + VPC endpoints/PrivateLink so traffic avoids the internet; KMS encryption; Secrets Manager.

Part 3 — Glossary of Terms

Core ML concepts

Supervised learning — training on labeled data (input → known output); classification & regression.

Unsupervised learning — finding structure in unlabeled data (clustering, dimensionality reduction, anomaly detection).

Semi-supervised learning — small labeled set + large unlabeled set (e.g. Ground Truth active learning).

Reinforcement learning — an agent learns by acting to maximize reward.

Classification / Regression — predict a discrete class vs a continuous number.

Clustering — group similar points without labels (K-Means).

Forecasting — predict future time-series values (DeepAR, Forecast).

Anomaly detection — flag rare/outlier points (Random Cut Forest).

Recommendation — predict user/item preferences (Factorization Machines, Personalize).

Feature / Label — an input variable vs the output being predicted.

Overfitting — memorizes training data, generalizes poorly. Fix: regularization, dropout, early stopping, more data, simpler model.

Underfitting — too simple; poor on train *and* test. Fix: more features/complexity, train longer.

Bias-variance tradeoff — high bias = underfit; high variance = overfit; seek the balance.

Regularization — penalize complexity to curb overfitting. **L1 (Lasso)** zeros weights (feature selection); **L2 (Ridge)** shrinks smoothly.

Dropout — randomly disable neurons during training (NN regularization).

Early stopping — halt when validation stops improving.

Epoch / Batch size / Learning rate — one full data pass · samples per weight update · step size for updates (too high diverges, too low stalls).

Gradient descent — optimization that moves weights to reduce loss (batch, SGD, mini-batch).

Loss/cost function — the quantity minimized (cross-entropy, MSE).

Hyperparameter vs parameter — set before training (learning rate, depth) vs learned during training (weights).

Transfer learning / Fine-tuning — reuse a pretrained model / continue training it on task data.

Ensemble / Boosting — combine models (bagging, boosting, stacking) / sequential error-correcting models (XGBoost).

Data leakage — outside/target info contaminates training and inflates scores (target leakage, test-into-preprocessing).

Train/validation/test split — partition to train, tune, and finally evaluate.

Cross-validation (k-fold) — train/validate over k folds for robust estimates.

Stratified sampling — preserve class proportions in splits.

Class imbalance — one class dominates; handle with SMOTE, resampling, class weights, right metric.

Curse of dimensionality / Dimensionality reduction — too many features hurts; reduce with PCA.

Data preparation & feature engineering

ETL / ELT — transform-then-load vs load-then-transform.

Batch vs streaming — bulk at intervals vs continuous real-time.

One-hot / Label encoding — categorical → binary columns vs categorical → integers (implies order).

Normalization / Standardization — Min-Max → [0,1] vs Z-score → mean 0, std 1.

Binning / Log transform — group continuous values into ranges / compress skew.

Imputation — fill missing values (mean/median/mode/model).

Outlier — extreme value; remove, cap (winsorize), or transform.

Tokenization / Embedding / Vectorization — split text into tokens / dense semantic vectors / convert data to numeric vectors.

TF-IDF — weights words by importance in a doc vs the corpus.

Feature engineering / selection — create/transform features / pick the most relevant.

Training/serving skew — features differ between training and inference; Feature Store prevents it.

Columnar formats (Parquet/ORC) — store by column; faster/cheaper for ML than CSV/JSON.

Data quality — validity, completeness, consistency (Glue Data Quality, Deequ).

Evaluation metrics

Confusion matrix — TP/FP/TN/FN table.

Accuracy — correct ÷ total; misleading on imbalance.

Precision — $TP \div (TP+FP)$; prioritize when false positives are costly.

Recall (Sensitivity/TPR) — $TP \div (TP+FN)$; prioritize when false negatives are costly (fraud, disease).

F1 — harmonic mean of precision & recall; good for imbalance.

Specificity (TNR) — $TN \div (TN+FP)$.

ROC / AUC — TPR vs FPR across thresholds; AUC summarizes ranking (1.0 perfect, 0.5 random).

PR-AUC — precision-recall AUC; better than ROC-AUC for heavy imbalance.

Threshold — probability cutoff for a class; tunes precision/recall.

RMSE / MAE / MAPE / R^2 — squared-error (penalizes big errors) / absolute error (robust) / percentage error / variance explained.

Log loss / cross-entropy — penalizes confident wrong probabilities.

SHAP — per-feature attribution of a prediction (used by Clarify).

Deployment & inference

Real-time / Serverless / Asynchronous / Batch transform — the four SageMaker inference modes (see Task 3.1).

Multi-model endpoint (MME) — many models behind one container to save cost.

Production variant / Shadow variant — weighted A/B traffic / mirrored traffic with responses discarded.

Deployment guardrails — blue/green, canary, linear traffic shifting with auto-rollback.

Auto scaling / Cold start — adjust instance count by target metric / latency spike when scaling up from zero.

Inferentia / Trainium / Graviton — AWS chips for cheap inference / cheap training / better price-perf CPU.

Latency vs throughput — per-request time vs requests per unit time.

Orchestration, IaC & CI/CD

CloudFormation / CDK — declarative IaC templates / IaC in a programming language.

CI/CD — automated build, test, deploy.

Automated retraining loop — drift → EventBridge → pipeline → evaluate → register → approve → deploy.

Model lineage — tracked history of data/code/steps that produced a model.

Monitoring, security & governance

Data drift vs Concept drift — input distribution shifts vs the input→output relationship changes.

Feature-attribution drift — the features driving predictions shift.

Baseline — reference stats (from training data) that monitoring compares against.

IAM / Execution role — identities & permissions / the role SageMaker assumes on your behalf.

Least privilege — grant only the permissions needed.

Identity- vs resource-based policy — attached to a principal vs to a resource (e.g. bucket policy).

VPC / VPC endpoint (PrivateLink) — isolated network / private path to AWS services without the internet.

Security group / NACL — instance-level vs subnet-level firewall.

KMS / Encryption at rest vs in transit — key management / protecting stored vs moving data.

Network isolation — training-job flag disabling outbound network access.

Responsible AI — fairness, explainability, transparency, governance (Clarify, Model Cards, Guardrails).

Compensatory scoring — overall pass; no per-domain minimum.

Generative AI

Foundation model (FM) / LLM — broad pretrained model / a text-focused FM.

Prompt engineering — craft inputs to steer output; cheapest customization, try first.

RAG (Retrieval-Augmented Generation) — retrieve documents into the prompt so the model uses fresh/proprietary knowledge without retraining (Bedrock Knowledge Bases).

Fine-tuning (GenAI) — train an FM on your data to adapt style/format/domain; choose over RAG for behavioral/format change, not just facts.

Vector database — stores embeddings for similarity search (OpenSearch, Aurora pgvector, Kendra).

Agents / Guardrails — FM orchestrating multi-step tool calls / policy filters for harmful content & PII.

Token / Context window / Temperature — text unit / max tokens considered / output randomness.

Hallucination — confident but false output; RAG & guardrails mitigate.

Distillation — train a smaller student to mimic a larger teacher.

Part 4 — AWS Service Reference (what it does · when to use)

SageMaker and its components (the core of the exam)

Amazon SageMaker (AI) — end-to-end managed ML platform. *Use for* building, training, tuning, deploying, and monitoring custom models.

SageMaker Studio — web IDE for the ML lifecycle. *Use as* the central workbench.

Data Wrangler — visual data prep, 300+ transforms. *Use when* cleaning/engineering features with minimal code.

Processing — run preprocessing/eval jobs at scale (sklearn/Spark). *Use for* scalable transforms and model evaluation steps.

Feature Store — online + offline feature repo. *Use to* reuse features and stop training/serving skew.

Ground Truth — managed labeling with active learning. *Use to* build labeled datasets.

Built-in algorithms — XGBoost, Linear Learner, K-Means, PCA, DeepAR, Random Cut Forest, etc. *Use when a* standard algorithm fits, to avoid custom code.

Script mode / BYOC — your training script in a managed container / your own Docker image. *Use for* custom frameworks / full control.

Managed Spot Training — spare capacity, big discount, needs checkpointing. *Use to* cut training cost.

Automatic Model Tuning (AMT) — hyperparameter optimization (Bayesian/Hyperband). *Use to* find best hyperparameters efficiently.

Autopilot / Canvas — AutoML / no-code ML. *Use for* quick baselines / business-analyst users.

JumpStart — pretrained models + solution templates. *Use to* start from a model instead of scratch.

Clarify — bias detection (pre/post-training) + SHAP explainability. *Use to* detect bias or explain predictions.

Debugger — inspect training tensors. *Use to* debug convergence, vanishing gradients, bottlenecks.

Experiments — track/compare runs. *Use for* reproducibility.

Model Registry — version & approve models with lineage. *Use to* manage model versions for audits and CI/CD.

Pipelines — native ML workflow orchestration. *Use as* the default ML CI/CD workflow tool.

Model Monitor — production drift detection (data quality, model quality, bias, feature-attribution). *Use to* catch drift in deployed models.

Inference Recommender — load-test to right-size endpoints. *Use to* pick instance type/size.

Neo — compile/optimize models for hardware/edge. *Use for* edge or hardware-specific speedups.

Model Cards / Model Dashboard / Role Manager — governance docs / central model view / scoped IAM roles. *Use for* responsible-AI governance and access setup.

Machine Learning / AI services

Bedrock — managed foundation models via API (+ Knowledge Bases/RAG, Agents, Guardrails). *Use for* GenAI apps without managing models.

Amazon Q — GenAI assistant (Business & Developer). *Use for* a managed assistant with minimal setup.

Comprehend — NLP (sentiment, entities, key phrases, PII). *Use to* analyze text with no model-building.

Comprehend Medical — clinical NLP (conditions, meds, PHI). *Use for* medical text extraction.

Rekognition — image/video analysis, moderation, faces, Custom Labels. *Use for* vision tasks off the shelf.

Textract — extract text/forms/tables from documents (beyond OCR). *Use to* digitize invoices/forms.

Transcribe — speech-to-text (custom vocab, diarization; Medical variant). *Use for* audio → text.

Polly — text-to-speech (neural voices, SSML). *Use for* generated speech.

Translate — neural translation. *Use for* language translation.

Lex — conversational bots (ASR + NLU). *Use to* build chatbots.

Kendra — intelligent enterprise search; can back RAG. *Use for* natural-language search over docs.

Personalize — real-time recommendations. *Use to* add recommendations without building a recommender.

Fraud Detector — managed fraud models. *Use to* detect fraud/fake accounts with little ML work.

Augmented AI (A2I) — human review of ML predictions. *Use for* human-in-the-loop review.

CodeGuru — ML code review (Reviewer/Security) + Profiler. *Use to* improve code quality/performance.

DevOps Guru — ML ops-anomaly detection. *Use for* ops anomaly alerts with no model-building.

HealthLake — FHIR health data lake + NLP. *Use for* health-data analytics/ML (niche).

Lookout for Equipment / Metrics / Vision — anomaly detection for sensors / business metrics / images. *Use for* targeted anomaly detection (recognize names).

Mechanical Turk — human-task marketplace. *Use as* a labeling workforce.

Analytics

Athena — serverless SQL on S3. *Use for* ad-hoc queries over S3 without a warehouse.

Data Firehose — managed streaming delivery (+ Lambda transform). *Use to* load streams into S3/Redshift/OpenSearch with no code.

Kinesis (Data Streams) — real-time streaming ingestion. *Use when* you build custom real-time consumers.

Kinesis Video Streams — stream video for analytics/ML. *Use for* video ingestion.

Managed Service for Apache Flink — real-time stream processing/analytics. *Use for* in-stream computations.

EMR — managed Spark/Hadoop. *Use for* heavy big-data processing beyond Glue.

Glue — serverless Spark ETL + Data Catalog + crawlers. *Use for* serverless ETL and schema cataloging.

Glue DataBrew — no-code visual prep. *Use for* point-and-click cleaning.

Glue Data Quality — data-quality rules. *Use to* validate datasets.

Lake Formation — data-lake governance/permissions. *Use for* fine-grained lake access control.

OpenSearch — search + log analytics; vector store. *Use for* search, observability, or RAG embeddings.

QuickSight — BI dashboards. *Use for* business-facing visualizations.

Redshift — data warehouse (Redshift ML trains from SQL). *Use for* large-scale analytical queries.

Storage

S3 — object storage / data lake (classes: Standard, Intelligent-Tiering, Glacier). *Use as* the default training-data store.

S3 Glacier — archival storage. *Use for* cheap long-term archive.

EBS — block storage on instances (gp3, io2/Provisioned IOPS). *Use for* instance-attached disks.

EFS — elastic shared NFS. *Use for* POSIX data shared across instances.

FSx (incl. Lustre) — high-throughput file systems; Lustre links to S3. *Use to* speed up training data loading.

Storage Gateway — hybrid on-prem ↔ AWS storage. *Use for hybrid access (niche).*

Databases

DynamoDB — serverless NoSQL key-value. *Use for low-latency lookups (e.g. online features).*

RDS — managed relational DB. *Use for transactional relational data.*

DocumentDB / Neptune / ElastiCache — MongoDB-compatible document DB / graph DB / in-memory cache. *Use for document, graph, and caching needs respectively.*

Compute & Containers

EC2 — virtual servers. *Use for custom compute / self-managed hosting.*

Lambda — serverless functions (<15 min). *Use for lightweight inference, triggers, glue logic.*

Batch — managed batch compute. *Use for large batch jobs.*

Serverless Application Repository — share/deploy serverless apps. *Use to reuse app templates (niche).*

ECR — container registry. *Use to store SageMaker/ECS/EKS images.*

ECS / EKS — container orchestration (AWS-native / Kubernetes). *Use to host models/services in containers.*

Developer Tools & IaC

CloudFormation — declarative IaC templates. *Use for reproducible infra.*

CDK — IaC in a programming language (synths to CFN). *Use when teams prefer code over YAML.*

CodePipeline / CodeBuild / CodeDeploy — CI/CD orchestration / build & test / deployment. *Use to automate build→test→deploy.*

CodeArtifact — artifact/package repository. *Use for managing build dependencies.*

X-Ray — distributed tracing. *Use to diagnose latency/perf across services.*

Application Integration & Orchestration

Step Functions — serverless workflow state machines (any service). *Use for orchestration including non-ML steps.*

MWAA (Managed Airflow) — managed Airflow DAGs. *Use when a team already uses Airflow.*

EventBridge — event bus / scheduled & event triggers. *Use to trigger pipelines/retraining on events.*

SNS / SQS — pub/sub notifications / message queues. *Use for fan-out alerts / decoupling and buffering.*

Management, Governance & Cost

CloudWatch (+ Logs) — metrics, logs, alarms, dashboards. *Use for monitoring endpoints/resources.*

CloudTrail — API-call audit log. *Use for governance/audit and retraining triggers.*

Config — resource compliance tracking. *Use to enforce/audit configuration rules.*

Compute Optimizer — right-sizing recommendations. *Use to optimize instance choices.*

Auto Scaling — scale resources to demand. *Use for elastic capacity.*

Systems Manager — ops management + Parameter Store (config/secrets). *Use for config storage and ops automation.*

Organizations / Service Catalog / Trusted Advisor / Chatbot — multi-account management / approved product catalog / best-practice checks (incl. cost) / ChatOps alerts.

Cost Explorer / Budgets / Billing and Cost Management — analyze spend / set alerts & quotas / manage billing. *Use for cost visibility and control.*

Networking & Security

VPC (+ endpoints / PrivateLink) — isolated network + private service access. *Use to keep traffic off the public internet.*

API Gateway — managed API front end. *Use to expose models/services as APIs.*

CloudFront — CDN. *Use for low-latency global content delivery.*

Direct Connect — dedicated on-prem ↔ AWS link. *Use for hybrid networking (niche).*

IAM — identities, roles, policies, groups. *Use for all access control (users, execution roles, least privilege, PassRole).*

KMS — encryption key management. *Use to encrypt S3, EBS, inter-container traffic.*

Macie — PII/PHI discovery in S3. *Use to find sensitive data for compliance.*

Secrets Manager — store/rotate secrets. *Use for DB credentials, API keys.*

Migration

DataSync — fast on-prem ↔ AWS data transfer. *Use to move large datasets into S3/EFS/FSx.*

Out-of-Scope Services — Don't Study, but Recognize as Distractors

Notable ML ones explicitly out of scope: **DeepRacer, HealthImaging, HealthOmics, Monitron, Panorama.**

Also out of scope: AppFlow, SWF, MQ; App Runner, Elastic Beanstalk, Lightsail, Outposts; CodeCatalyst, CloudShell, Application Composer; the entire IoT family; most security services beyond IAM/KMS/Macie/Secrets Manager (Cognito, GuardDuty, Inspector, Security Hub, WAF, Shield, Detective, ACM, CloudHSM); the Elemental/Media family; all Business/End-User computing apps. Spotting the out-of-scope list is a fast way to eliminate wrong answers.

How to Study This

For every task statement, know *what each item is* and *when it's the right choice* — the exam tests the second far more. Train on qualifier words: **"least operational overhead"** → managed AI service/serverless · **"most cost-effective"** → Spot, serverless/async, batch, multi-model, Savings Plans · **"lowest latency"** → real-time endpoint / online Feature Store · **"must not traverse the internet"** → VPC endpoints/PrivateLink · **"detect drift"** → Model Monitor · **"explain predictions / detect bias"** → Clarify.

With a DevSecOps/infrastructure background, Domains 3–4 (orchestration, IaC, containers, VPC, IAM, cost) are mostly vocabulary mapping onto what you already do — the least-privilege/`PassRole`, VPC-endpoint, and KMS material especially. Spend extra time on Domain 2's ML theory (metrics, algorithm selection, regularization, tuning), the least infrastructure-shaped part of the exam.